

DateDimension	
PK	<u>DATESK</u>
	<u>FULLDATE</u>
	DAY
	MONTH
	YEAR

1
Many

FactSales	
PK	SALESSK
FK	PRODUCTSK
FK	CUSTOMERSK
FK	<u>DATESK</u>
	QUANTITY
	SALEPRICE
	TOTALREVENUE
	GROSSMARGIN
	SOURCETABLE

Many

Many

CustomerDimension	
PK	<u>CUSTOMERSK</u>
	<u>CUSTOMERID</u>
	CUSTOMERNAME
	MYIC
	GENDER
	SOURCETABLE
	REGIONNAME

1

ProductDimension	
PK	<u>PRODUCTSK</u>
	<u>PRODUCTID</u>
	PRODUCTNAME
	PURCHASECOST
	SOURCETABLE

1

DESIGN JUSTIFICATION:

For the data warehouse design, I decided to use the star schema approach because it is honestly the most practical and straightforward way to structure analytical data, especially when the goal is to make reporting and decision-making faster and easier. In simple terms, the star schema places the FactSales table in the center, and this table stores all the numerical information that the business will usually analyse, like quantity sold, sale price, total revenue, and gross margin. If you look closely at the schema, you will see three dimension tables surrounding the fact table, which are CustomerDimension, ProductDimension, and DateDimension. These tables basically act as detailed references that describe the sales from different points of view, and this makes it easier to break down sales by customer, product, and time.

One important thing that you can see immediately is that each dimension uses a surrogate key such as CustomerSK or ProductSK instead of relying on natural keys from the original databases. The reason this is important is because Vertex is using two different source databases, and natural keys from different systems may clash or change unexpectedly. So, by using surrogate keys, the warehouse remains stable and avoids errors that come from differences in key formats or source system changes. Another thing I purposely included is the "SourceTable" field in every table. This might look like a small addition, but it actually helps a lot because it tells you where each record originally came from. If something goes wrong during ETL or data validation, you can trace the data back to the exact source table. This makes debugging and quality checking much easier.

In the CustomerDimension, I added RegionName because it allows the business to group customers by region, and this becomes useful when they want to compare sales performance between different geographic areas. This also prepares the warehouse for future marketing segmentation since region-based analysis is very common in business intelligence. The ProductDimension includes key attributes like ProductID, ProductName, PurchaseCost, and SourceTable. This structure helps the company understand product profitability and compare performance between different product categories.

Meanwhile, the DateDimension stores FullDate, Day, Month, and Year, which removes the need to manually break down dates every time a report is generated. Time-based analysis becomes much simpler and faster. The overall design follows the standard star schema principle where dimensions are denormalized for readability and speed, while the fact table remains narrow and focused on numerical measures. This combination helps analytical tools work efficiently without having to join too many tables. In short, the design makes sure that data from both operational databases is combined in a clean and organized way, while also giving the company flexibility to expand the warehouse in the future. It ensures stability through surrogate keys, supports traceability with the SourceTable field, and covers multiple analysis perspectives through its dimensions. Everything is arranged to support faster reporting, easier data exploration, and long-term maintainability.

SCALABILITY AND FUTURE CONSIDERATIONS

When planning for scalability and future needs, I made sure that this warehouse structure can grow naturally without causing issues later. The fact table is designed to handle very large volumes because it mainly stores numeric values and foreign keys, which means it stays compact and efficient even if the company collects hundreds of thousands or millions of sales records over the years. If sales volume increases significantly in the future, the company can apply partitioning techniques, especially on the DateSK column, to make queries and data loading much faster.

The dimension tables are also built with flexibility in mind. For example, the CustomerDimension includes RegionName now, but if the business decides later to track other details like customer loyalty tier, spending category, or even preferred shopping method, these can be added without affecting the structure of the fact table. The same goes for the ProductDimension. Right now, it stores basic product details such as name and purchase cost, but it can easily be expanded to include product categories, suppliers, brands, or warranty information. Since everything is tied through surrogate keys, the warehouse will not break if the original source systems undergo changes such as new ID formats or new business rules.

Another major scalability point is the ability to implement Slowly Changing Dimensions when needed. If a customer moves to a new region, or if the company updates product pricing, the warehouse can keep historical versions of these records without losing old information. This is very important for long-term trend analysis. For future data integration, the warehouse is also ready to handle new sources. Since every table includes the SourceTable field, adding a new POS system, e-commerce platform, or ERP system later will not cause confusion because each record can still be traced back to its origin. This also makes the ETL process easier to maintain. The design supports advanced analytics as well. If the business wants to introduce forecasting models, customer segmentation, or predictive analytics, the warehouse structure already provides clean and organized data that can be used directly for machine learning.

The DateDimension can also be expanded later with fiscal calendars, promotional period tags, and holiday indicators, which will help improve the accuracy of time-based analysis. Another thing to consider as the system grows is security, especially for sensitive customer details like MYIC. In the future, data masking, encryption, or role-based access can be implemented without needing to change the schema. If the business outgrows on-premise hardware, the warehouse can also be migrated to cloud platforms such as AWS Redshift or Google BigQuery with minimal changes because the star schema is compatible with these technologies. Overall, the design is made to support long-term growth. It allows the company to expand the number of attributes, add new analysis capabilities, integrate more data sources, and maintain historical accuracy and all of them will not require the system to be restructured from scratch without restructuring the system from scratch.